

An Introduction to Regression Neural Networks

Moaaz Abo-Shaeer

0 Background

"AI" and Machine learning are quite hot topics, and it may not be obvious what they actually are. Machine learning is often classified into *supervised* and *unsupervised* learning. Supervised algorithms learn from data that has been labeled¹ by humans, while unsupervised algorithms attempt to find patterns on their own.

Supervised machine learning can be classified into *classification* and *regression*. Classification is where the algorithm is given data and asked to classify it into discrete categories (e.g. photos in your phone classified into folders for each person), while regression is basically function approximation, where an algorithm is given data points and provides the general trend that this data follows allowing it to predict values for new data points. This introduction focuses on neural networks used in regression.

1 Simple Linear Regression

If we know that our data has a linear trend then we know it must follow the equation $y = wx + b$, our task now is simply to find w and b , called the **weight** and **bias** respectively. There is an analytical solution to this specific problem, but it doesn't scale to other cases so we will use a more numerical solution that can pave the way to neural networks. We must first find a metric to evaluate our regression line which we'll refer to as the *loss function*, our metric will be the mean squared error (MSE).

Definition 1. Mean Squared Error is defined as:

$$E = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{n} \sum_{i=1}^n (y_i - (wx_i + b))^2$$

Where y_i is an actual data point and $\hat{y}_i = wx_i + b$ is the prediction of our model.

Now we can tweak w and b and then evaluate the model using MSE, by repeating these two steps, we can minimize the error of our model.

1.1 Gradient Descent

Let Δw and Δb be the tweaks we apply to our parameters. Since we've used MSE, we know that the error follows parabolic curves with varying w or b . So by observing the rate of change of the error function with respect to our parameters we find that the further we are from the minimum error the greater the slope is, which can be used to indicate *how much* we should tweak our parameter. We also find that if the rate of change is positive, then we must decrease our parameter (w or b) to move towards the minimum and vice versa. Therefore, $\Delta w \propto -\frac{\partial E}{\partial w}$ and $\Delta b \propto -\frac{\partial E}{\partial b}$. Knowing this, we can tweak our parameters for multiple iterations to minimize error.

$$w' = w + \Delta w = w - \eta \frac{\partial E}{\partial w} = w + \frac{2\eta}{n} \sum_{i=1}^n (y_i - (wx_i + b))x_i$$
$$b' = b + \Delta b = b - \eta \frac{\partial E}{\partial b} = b + \frac{2\eta}{n} \sum_{i=1}^n (y_i - (wx_i + b))$$

Where η is a constant of proportionality called the *learning rate*.

And the iterative algorithm we just described is called *gradient descent*, where each iteration over the data is called an *epoch*.

¹ *Labeled* data is data that's associated with a correct answer, e.g., images of people which have the labels "male" or "female" provided to a model which learns to differentiate men from women.

Example 1 (Simple Linear Regression Code). A simple example code of linear regression using gradient descent.

```
import matplotlib.pyplot as plt
import numpy as np

X = np.linspace(-10, 10, 50) # 50 evenly spaced numbers between -10 and 10
Y = 0.3 * X + 1 + np.random.normal(0, 1, size=X.shape) # y = 0.3x + 1 + noise

w = np.random.rand()
b = np.random.rand()

eta = 2e-4 # Learning rate
for epoch in range(3):
    w += 2 * eta * np.sum((Y - (w * X + b)) * X)
    b += 2 * eta * np.sum(Y - (w * X + b))

Yhat = w * X + b # Predictions

print(f'Final weights: w = {w}, b = {b}')
```

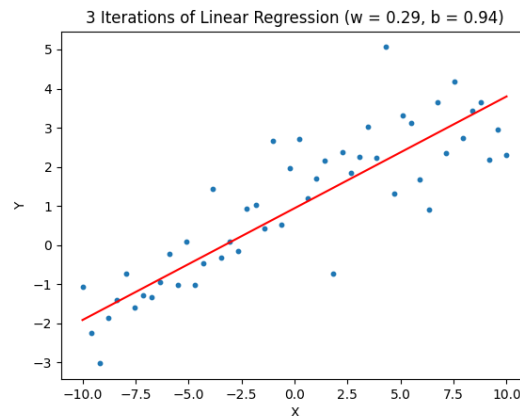


Figure 1: The original data points (blue) following the trend $y = 0.3x + 1$, and the regression line (red) after 3 iterations of gradient descent over the data set.

2 More Complex Models

Previously, we only discussed scalar single variable models used for regression, and the solution we found may have seemed overkill for the a problem with a known analytical solution. But we can apply the same method to models with multiple inputs and outputs as discussed in the next sections.

2.1 Multivariable Linear Regression

If the value we are trying to predict depends linearly on multiple variables, our model becomes $y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$. But we can still use gradient descent to minimize error.

$$w'_k = w_k + \Delta w = w_k - \eta \frac{\partial E}{\partial w} = w_k + \frac{2\eta}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_{k,i}$$

$$b'_k = b_k + \Delta b = b_k - \eta \frac{\partial E}{\partial b} = b_k + \frac{2\eta}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

Notice that i is the index of the data point while k is the index of the term in the model.

2.2 Vector-valued Linear Regression

In the previous section, we considered a model with multiple inputs. Now we can find a Model with n inputs and m outputs called vector-valued linear regression².

$$\begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_m \end{bmatrix} = \begin{bmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,n} \\ w_{2,1} & w_{2,2} & \dots & w_{2,n} \\ \dots & \dots & \dots & \dots \\ w_{m,1} & w_{m,2} & \dots & w_{m,n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \dots \\ b_m \end{bmatrix}$$

Notice that for a single y_i the model has the same equation as multivariable linear regression, it's only more concise to use linear algebra in this case, we can write the same equation in much shorter form.

$$\vec{y} = W\vec{x} + \vec{b}$$

2.3 Polynomial Regression

If the data doesn't follow a linear trend, we must use a non-linear model to approximate it. And the easiest non-linear model is an n th-degree polynomial $y = b + w_1x + w_2x^2 + \dots + w_nx^n$. We can still use MSE and gradient descent, but the gradient descent equations for the weights are slightly different.

$$w'_k = w_k + \Delta w = w_k - \eta \frac{\partial E}{\partial w} = w_k + \frac{2\eta}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \cdot kx^{k-1}$$

3 Regression Neural Networks

To approximate non-linear data, we can use multiple stages of vector-valued linear regression with some non-linear function added between them. That is the basic structure of a regression neural network.

3.1 ReLU and Activation Functions

If our model only consists of multiple stages of linear functions it will also produce a linear function, so if we want the model to approximate non-linear data we must add a non-linear function between these stages. Activation functions are these non-linear functions, the most common example for regression is ReLU (Rectified Linear Unit) which is just a ramp function $ReLU(x) = \max(0, x)$. But the linear transformations preceding and following it can bend it to fit the data. Let's use simple linear equations as an example.

Example 2 (ReLU Example). If the first stage is $z = w_1x + b_1$, the second stage is the ReLU function $a = ReLU(z) = \max(0, z)$, and the third stage is $\hat{y} = w_2a + b_2$, then the full model is

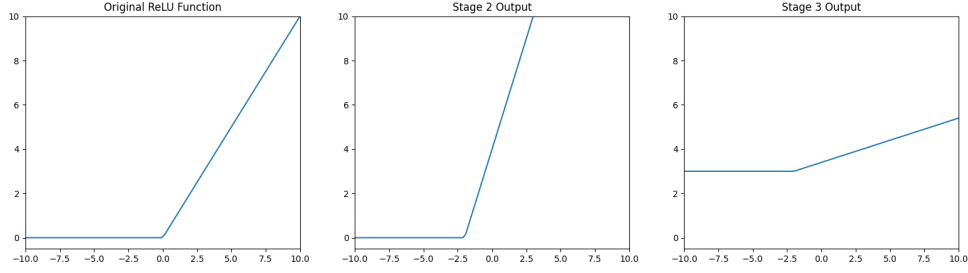
$$\hat{y} = w_2 ReLU(w_1x + b_1) + b_2$$

Notice that w_1 produces x axis scaling, b_1 produces x axis translation, w_2 produces y axis scaling, and b_2 produces y axis translation. These parameters can optimized to make the model fit the data, and if we can add multiple similar functions we can produce more complex non-linear models.

The stages in a neural network are called *layers*³, and the internal stages which produce z and a are called *hidden layers*.

²Also called multivariate linear regression, but I used this name to prevent confusion

³The first and second stage are considered one layer since applying the activation function isn't usually a separate stage



3.2 The Structure of a Neural Network

Let us consider a general neural network with multiple inputs in an input vector called *the input layer* $\vec{x}$, the first layer consists of a weight matrix W^1 , a bias vector $\vec{b}^1$ and an activation function f^1 .

$$\vec{a}^1 = f^1(\vec{z}^1) = f^1(W^1\vec{x} + \vec{b}^1)$$

The vector $\vec{a}^1$ is called *activation* and it's the output of the first layer, and the vector $\vec{z}^1$ is called *pre-activation* and it's the output of the linear transformation before the activation function. The second layer takes $\vec{a}^1$ as input and has a similar equation.

$$\vec{a}^2 = f^2(\vec{z}^2) = f^2(W^2\vec{a}^1 + \vec{b}^2)$$

In a classification neural network, the output usually uses an activation function like softmax or sigmoid. But in regression the output layer may not have an activation function at all.

$$\hat{y} = W^3\vec{a}^2 + \vec{b}^3$$

3.3 Backpropagation

Notice that in **Example 2** that the presence of the activation function will make the calculation of $\frac{\partial E}{\partial w_i}$ or $\frac{\partial E}{\partial b_i}$ more complex. So we will have to use the chain rule to get the partial derivatives of the error with respect to the parameters, applying the chain rule backward throughout the whole model is the *backpropagation* algorithm.

3.3.1 Hadamard Product

Multiplying two vectors element-wise is called the *Hadamard product* and is denoted by $\odot$, where the main difference between it and the dot product is that the output of the Hadamard product is also a vector.

$$\begin{bmatrix} a_1 \\ a_2 \\ \dots \\ a_n \end{bmatrix} \odot \begin{bmatrix} b_1 \\ b_2 \\ \dots \\ b_n \end{bmatrix} = \begin{bmatrix} a_1 b_1 \\ a_2 b_2 \\ \dots \\ a_n b_n \end{bmatrix}$$

Example 3 (Backpropagation Example). There are quick formulas to apply backpropagation automatically, but we will first do it by hand on a small neural network as an example.

Using the standard notation, we place the layer number as a superscript on the parameters.

In a two-layer neural network, if the first layer is

$$\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} w_1^1 \\ w_2^1 \end{bmatrix} x + \begin{bmatrix} b_1^1 \\ b_2^1 \end{bmatrix} \quad (1)$$

with the ReLU function applied to it

$$\begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = ReLU \left(\begin{bmatrix} z_1^1 \\ z_2^1 \end{bmatrix} \right) = \begin{bmatrix} \max(0, z_1) \\ \max(0, z_2) \end{bmatrix} \quad (2)$$

and the second (layer without an activation function) is

$$\hat{y} = \begin{bmatrix} w_1^2 & w_2^2 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} + b_1^2 = w_1^2 a_1 + w_2^2 a_2 + b_1^2 \quad (3)$$

We need to differentiate the error with respect to the parameters, and we know that $\frac{\partial E}{\partial w_i^2} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_i^2}$ and $\frac{\partial E}{\partial b_1^2} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b_1^2}$, but $\frac{\partial \hat{y}}{\partial b_1^2} = 1$.

For the weights, we use the chain rule on the error function to get $\frac{\partial E}{\partial \hat{y}} = \frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$. Then, from **Eq. (3)** we find that $\frac{\partial \hat{y}}{\partial w_i^2} = a_i$. Therefore,

$$\begin{bmatrix} \frac{\partial E}{\partial w_1^2} \\ \frac{\partial E}{\partial w_2^2} \end{bmatrix} = \frac{\partial E}{\partial \hat{y}} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}, \quad \frac{\partial E}{\partial b_1^2} = \frac{\partial E}{\partial \hat{y}} \quad (4)$$

From **Eq. (1)** and **Eq. (2)** we know that $\frac{\partial E}{\partial w_i^1} = \frac{\partial E}{\partial a_i} \cdot \frac{\partial a_i}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_i^1}$. Then by looking at **Eq. (3)**, $\frac{\partial E}{\partial a_i} = \frac{\partial E}{\partial \hat{y}} \cdot w_i^2$, from **Eq. (2)** we find that $\frac{\partial a_i}{\partial z_i} = \text{ReLU}'(z_i)$, and from **Eq. (1)** we find that $\frac{\partial z_i}{\partial w_i^1} = x$. Therefore,

$$\begin{bmatrix} \frac{\partial E}{\partial w_1^1} \\ \frac{\partial E}{\partial w_2^1} \end{bmatrix} = \frac{\partial E}{\partial \hat{y}} \begin{bmatrix} w_1^2 \\ w_2^2 \end{bmatrix} \odot \begin{bmatrix} \text{ReLU}'(z_1) \\ \text{ReLU}'(z_2) \end{bmatrix} \cdot x, \quad \begin{bmatrix} \frac{\partial E}{\partial b_1^1} \\ \frac{\partial E}{\partial b_2^1} \end{bmatrix} = \frac{\partial E}{\partial \hat{y}} \cdot \begin{bmatrix} w_1^2 \\ w_2^2 \end{bmatrix} \cdot \begin{bmatrix} \text{ReLU}'(z_1) \\ \text{ReLU}'(z_2) \end{bmatrix} \quad (5)$$

Where the derivative of the ReLU function $\text{ReLU}'(z_i) = \begin{cases} 1 & z_i > 0 \\ 0 & z_i \leq 0 \end{cases}$.

3.4 Generalised Backpropagation Formulas⁴

Now we can look at the standard formulas of backpropagation. Consider a neural network with L layers, with weight matrices W^l and bias vectors $\bar{b}^l$, each layer with an activation function f^l , and let z_j^l and a_j^l be the pre-activations and activations respectively. The vector δ^l contains the partial derivatives of the error with respect to the pre-activations of layer l , which means that

$$\delta_j^l = \frac{\partial E}{\partial z_j^l} \quad (6)$$

Therefore, the vector δ^L of the last layer is

$$\delta^L = \frac{\partial E}{\partial a^L} \odot \frac{\partial a^L}{\partial z^L} = \frac{\partial E}{\partial a^L} \odot f'^L(z^L) \quad (7)$$

For the hidden layers we have

$$\delta^l = (W^{l+1})^T \delta^{l+1} \odot f'^l(z^l) \quad (8)$$

And the partial derivatives of the error with respect to the weights and biases of each layer are

$$\frac{\partial E}{\partial w_{j,k}^l} = \frac{\partial E}{\partial z_j^l} \cdot (a_k^{l-1}), \quad \frac{\partial E}{\partial b_j^l} = \frac{\partial E}{\partial z_j^l}$$

And the vector form of these equations is as follows

$$\frac{\partial E}{\partial W^l} = \delta^l (a^{l-1})^T, \quad \frac{\partial E}{\partial \bar{b}^l} = \delta^l$$

⁴These formulas will not be properly explained here, if you want a better understanding of them I would recommend reading Chapter 2 of Neural Networks and Deep Learning by Michael Nielsen and watch Backpropagation, intuitively by 3Blue1Brown for intuition.